

installations that will be used to load balance the Web servers located in the second layer (Layer 2). In Layer 2, I will set up two *nginx* installations that serve websites, including PHP pages. Instead of two, you can of course have as many Web servers as required. The number of Web servers should be dependent on the total load on the servers. If you feel that the load on the servers is increasing, you can easily add another Web server to the cluster.

In addition to this, there can be a third layer (Layer 3) of database servers. The database is typically used by the applications running on the Web/application servers, where the application directly makes a call to the database using an appropriate method. Database clustering has a slightly different approach and we will take up this subject at a later date. Also, the database layer is independent of our current configuration, and can be added and removed at will without impacting the current set-up.

The installation method of *nginx* is identical on all the four servers in our set-up. From the configuration perspective, the Layer 1 reverse-proxy servers will have identical active-passive configurations and the Layer 2 Web servers will have identical configurations.

The Debian Etch has quite a dated version of *nginx*, but it is good enough for our purposes. So I have decided to stick with this old but stable version.

In order to install *nginx* in Debian Etch, issue the following command as the root user:

```
root@all-servers # apt-get install nginx
```

Configuring *nginx* on Web Servers (Layer 2)

We will start by configuring *nginx* Web servers located in Layer 2 and configuring PHP5 support in it using FastCGI. I will configure *nginx* on one Web server (server1) and the second Web server (server2) will have an identical configuration. We look at the configuration files for server2 later.

Let us first back-up the default configuration file for *nginx*:

```
root@server_1 # cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.orig
```

Now, create a new */etc/nginx/nginx.conf* file with the following data:

Round-robin DNS

Round-robin DNS is a technique to balance load across multiple servers by providing multiple IP addresses in response to a request for domain resolution. This is typically done by creating multiple A records in the DNS server for a domain. The actual load balancing depends on how the client responds to the response returned by the name server. When a client requests for name resolution, the DNS server responds with a list of redundant IP addresses. Sometimes the resolver tries to arrange the IP address list to give priority to the numerically closer network. A few clients pick the first IP address from the list and a few others may try out alternate addresses in case the first one fails.

RR DNS should not be used as the only method of load balancing as it suffers from a drawback of IP address caching and reuse, both within the DNS hierarchy and also at the client site. While responding to a request the DNS server does not consider the geographically nearer location, network congestion, server load and transaction time, etc.

This is best used when you have uniformly distributed data centres across various geographies. This is also typically used to balance the traffic across multiple data centres within the same geography.

```
user www-data;
worker_processes 1;

error_log /var/log/nginx/error.log;
pid /var/run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    access_log /var/log/nginx/access.log;
    sendfile on;
    keepalive_timeout 65;
    tcp_nodelay on;
    gzip on;
```

Enterprise Mail Server, Linux SBS Server,
Anti SPAM, Antivirus and HTTP Filtering.

Bandwidth Management, Internet Access Control,
Content Filtering, Web Access Reporting.

Customised Linux Product Development.

TechnoInfotech™

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Tel.: 91-22-6633 8900 Ext. 324. info@technoinfotech.com